



nice, but for now.

am still doing authentication set up.

don't mke plan for product and so on, tell a plan to that state how it would be handle.

for now for the branch dashboard, we are just going to have, welcome to branch in demo view, not product management or anything concerning inventory yet

Perfect! Let me create a focused implementation plan for **Branch Management during Authentication/Onboarding phase** - just the basics, no inventory features yet.

Implementation Plan: Branch Management (Auth & Onboarding Phase)

Overview

Objective: Implement basic branch management as part of the onboarding flow. Users can create branches during organization setup, switch between branches, and see a simple branch dashboard (demo view only).

Timeline: 1 week

Priority: Medium (required for multi-location onboarding)

Scope: Branch CRUD, branch switching, welcome dashboard (NO inventory, products, or stock yet)

User Journey: App → Organization → Branch

Flow Diagram

1. User completes authentication
↓
2. Welcome Screen: "What would you like to do?"
 - Create an organization
 - Join an organization
 - Explore Orvionhub↓
3. User selects "Create an organization"
↓
4. Organization Setup Wizard
 - Step 1: Organization Details (name, type, country, currency)
 - Step 2: Branch Setup (create first branch)

- Step 3: Select Applications (Inventory, Tasks, etc.)
Step 4: Choose Plan (Free/Standard/Premium)
↓
5. Branch Context Established
- Active Organization: "Code X Stores"
 - Active Branch: "Main Store" (primary)
- ↓
6. Redirect to Branch Dashboard
- Shows: "Welcome to Main Store"
 - Demo content only (no inventory yet)

Technical Context Flow

```
// After authentication
{
  user: {
    id: "user_123",
    email: "code@example.com",
    name: "Code X"
  },

  // After organization creation
  activeOrganization: {
    workspaceId: "ws_456",
    name: "Code X Stores Ltd",
    plan: "free"
  },

  // After branch setup
  activeBranch: {
    branchId: "branch_789",
    name: "Main Store",
    isPrimary: true,
    workspaceId: "ws_456"
  }
}

// Every request includes:
headers: {
  "x-workspace-id": "ws_456",
  "x-branch-id": "branch_789"
}
```

Phase 1: Database Schema & Branch CRUD (Days 1-3)

User Story 1.1: Branch Data Model

As a system architect, I need a simple branch data model so organizations can have multiple locations during onboarding.

Acceptance Criteria

- [] branches table created in Convex with minimal fields:
 - `_id`
 - `workspaceId` (FK to workspaces)
 - `name` (e.g., "Main Store", "Warehouse")
 - `code` (short identifier, e.g., "MAIN", "WH")
 - `isPrimary` (boolean, one per workspace)
 - `status` (active or archived)
 - `createdAt`, `updatedAt`
- [] Indexes: `by_workspace`, `by_workspace_status`, `by_workspace_primary`
- [] Every branch belongs to exactly one workspace
- [] One branch per workspace marked as `isPrimary: true`
- [] Branch code unique within workspace

Technical Tasks

- [] Add branches table to `convex/schema.ts`
- [] Create indexes for efficient queries
- [] Migration script: create primary branch for existing workspaces
- [] Validation: only one `isPrimary` branch per workspace

Convex Schema

```
// convex/schema.ts

branches: defineTable({
  workspaceId: v.id("workspaces"),
  name: v.string(),
  code: v.string(),
  isPrimary: v.boolean(),
  status: v.union(v.literal("active"), v.literal("archived")),
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])
.index("by_workspace_status", ["workspaceId", "status"])
.index("by_workspace_primary", ["workspaceId", "isPrimary"]);
```

User Story 1.2: Branch Creation During Onboarding

As a new user creating an organization, I want to set up my first branch so I can start using the application.

Acceptance Criteria

- [] Organization setup wizard includes "Branch Setup" step
- [] Default branch name: "Main Store" (editable)
- [] Auto-generate branch code from name (e.g., "Main Store" → "MAIN")
- [] First branch automatically marked as `isPrimary: true`
- [] Cannot skip branch creation (required step)
- [] Success message: "Branch 'Main Store' created successfully"
- [] After creation, redirect to branch dashboard

Technical Tasks

- [] Frontend: Branch setup step in onboarding wizard
- [] Backend: POST `/v1/workspaces/:workspaceId/branches`
- [] Convex mutation: `branches.create`
- [] Auto-generate branch code
- [] Set `isPrimary: true` for first branch

Onboarding Wizard Step

```
// Step 2: Branch Setup
function BranchSetupStep() {
  const [branchName, setBranchName] = useState("Main Store");
  const [branchCode, setBranchCode] = useState("MAIN");

  // Auto-generate code from name
  const generateCode = (name: string) => {
    return name.toUpperCase().replace(/^[A-Z]/g, '').slice(0, 4);
  };

  return (
    <div>
      <h2>Set Up Your First Location</h2>
      <p>Where will you operate from?</p>

      <Input
        label="Branch Name"
        value={branchName}
        onChange={(e) => {
          setBranchName(e.target.value);
          setBranchCode(generateCode(e.target.value));
        }}
        placeholder="e.g., Main Store, Warehouse, Office"
      />
    </div>
  );
}
```

```

    />

    <Input
      label="Branch Code"
      value={branchCode}
      onChange={(e) => setBranchCode(e.target.value.toUpperCase())}
      placeholder="e.g., MAIN, WH, OFF"
      maxLength={4}
    />

    <InfoBox>
      This will be your primary branch. You can add more branches later.
    </InfoBox>

    <Button onClick={handleCreate}>Continue</Button>
  </div>
);
}

```

User Story 1.3: Branch List & Selection

As a user, I want to see all branches in my organization and switch between them so I can work in different locations.

Acceptance Criteria

- [] Branch selector dropdown in app header (after organization selector)
- [] Shows all active branches in current organization
- [] Current branch highlighted
- [] Primary branch marked with "Primary" badge
- [] Branches sorted alphabetically
- [] Clicking a branch switches context immediately
- [] Page reloads with new branch data
- [] Empty state if no branches (should not happen)

Technical Tasks

- [] Frontend: Branch selector component (dropdown)
- [] Query: `branches.getByWorkspace({ workspaceId })`
- [] State management: update `activeBranch` in global store
- [] Refetch branch-scoped data on branch change
- [] Preserve organization context when switching branches

Branch Selector Component

```
// components/BranchSelector.tsx

export function BranchSelector() {
  const { activeOrganization } = useAuthStore();
  const { activeBranch, setActiveBranch } = useBranchStore();
  const branches = useQuery(api.branches.getByWorkspace, {
    workspaceId: activeOrganization.workspaceId
  });

  const handleSelect = (branch: Branch) => {
    setActiveBranch(branch);
    // Reload app with new branch context
    window.location.reload();
  };

  return (
    <Dropdown>
      <DropdownTrigger>
        <div className="flex items-center gap-2">
          <BuildingIcon className="w-4 h-4" />
          <span>{activeBranch?.name || "Select Branch"}</span>
          {activeBranch?.isPrimary && (
            <Badge variant="secondary">Primary</Badge>
          )}
        </div>
      </DropdownTrigger>

      <DropdownContent>
        {branches?.map((branch) => (
          <DropdownItem
            key={branch._id}
            onClick={() => handleSelect(branch)}
            className={activeBranch?._id === branch._id ? "bg-gray-100" : ""}
          >
            <div className="flex items-center justify-between">
              <span>{branch.name}</span>
              {branch.isPrimary && (
                <Badge variant="outline">Primary</Badge>
              )}
            </div>
          </DropdownItem>
        ))}
      </DropdownContent>
    </Dropdown>
  );
}
```

User Story 1.4: Branch Edit (Basic)

As an admin, I want to edit my branch name so I can correct mistakes or update the location name.

Acceptance Criteria

- ☐ Branch settings page has "Edit Branch" button
- ☐ Edit modal pre-fills with current branch name
- ☐ Can update: branch name, branch code
- ☐ Cannot change `isPrimary` flag
- ☐ Cannot change workspace association
- ☐ Validation: branch code unique within workspace
- ☐ Success message on update
- ☐ Changes reflect immediately in branch selector

Technical Tasks

- ☐ Frontend: Branch edit modal
- ☐ Backend: `PATCH /v1/workspaces/:workspaceId/branches/:branchId`
- ☐ Convex mutation: `branches.update`
- ☐ Unique code validation
- ☐ Audit log: `branch.updated`

User Story 1.5: Branch Context Middleware

As a system, I need to validate branch access on every request so users can only access branches in their organization.

Acceptance Criteria

- ☐ Middleware `requireBranchAccess` added to all branch-scoped routes
- ☐ Validates:
 1. User is authenticated
 2. User belongs to workspace
 3. Branch exists in workspace
- ☐ Returns 403 if branch not found
- ☐ Error message: "Branch not found or you don't have access"
- ☐ Middleware extracts `branchId` from request headers
- ☐ Branch context attached to request object

Technical Tasks

- [] Create backend/src/middleware/requireBranchAccess.ts
- [] Add to branch-scoped routes (for now, just dashboard)
- [] Extract x-branch-id header
- [] Attach request.branchId for handlers

Middleware Implementation

```
// backend/src/middleware/requireBranchAccess.ts

import { FastifyRequest } from 'fastify';

export async function requireBranchAccess(request: FastifyRequest, reply: any) {
  const user = request.user;
  const workspaceId = request.headers['x-workspace-id'] as string;
  const branchId = request.headers['x-branch-id'] as string;

  if (!workspaceId) {
    return reply.code(400).send({ error: 'Workspace ID required' });
  }

  if (!branchId) {
    return reply.code(400).send({ error: 'Branch ID required' });
  }

  // Validate user belongs to workspace
  const membership = await request.convex.query('workspaceMemberships.getByWorkspaceId', {
    workspaceId,
    userId: user.id
  });

  if (!membership) {
    return reply.code(403).send({ error: 'You do not have access to this organization' });
  }

  // Validate branch exists in workspace
  const branch = await request.convex.query('branches.getById', {
    branchId,
    workspaceId
  });

  if (!branch) {
    return reply.code(403).send({ error: 'Branch not found or you do not have access' });
  }

  // Attach branch context to request
  request.branchId = branchId;
  request.workspaceId = workspaceId;
}
```


Phase 2: Branch Dashboard - Demo View (Days 4-7)

User Story 2.1: Branch Dashboard (Welcome View)

As a user who just created a branch, I want to see a welcome dashboard so I know I'm in the right place and what to do next.

Acceptance Criteria

- [] Dashboard shows branch name prominently: "Welcome to Main Store"
- [] Shows branch info: code, status (Primary)
- [] Shows organization name: "Part of Code X Stores Ltd"
- [] Demo content only (no real inventory data)
- [] Placeholder sections:
 - "Products: Coming Soon"
 - "Sales: Coming Soon"
 - "Stock: Coming Soon"
 - "Reports: Coming Soon"
- [] "What's Next?" section with onboarding steps:
 - Add products
 - Set up staff
 - Record your first sale
- [] Branch selector visible to switch branches

Technical Tasks

- [] Frontend: Branch dashboard component
- [] Query: `branches.getById({ branchId })`
- [] Query: `workspaces.getById({ workspaceId })`
- [] Demo/placeholder UI components
- [] Onboarding steps component

Branch Dashboard Component

```
// surfaces/inventory/pages/BranchDashboard.tsx

export function BranchDashboard() {
  const { activeBranch, activeOrganization } = useBranchStore();
  const { user } = useAuthStore();

  if (!activeBranch) {
    return <div>Loading...</div>;
  }
}
```

```

}

return (
  <div className="p-6">
    {/** Header */}
    <div className="mb-6">
      <h1 className="text-2xl font-bold">
        Welcome to {activeBranch.name}
      </h1>
      <p className="text-gray-600">
        Part of {activeOrganization.name}
      </p>
      <div className="flex gap-2 mt-2">
        <Badge>{activeBranch.code}</Badge>
        {activeBranch.isPrimary && <Badge variant="secondary">Primary</Badge>}
      </div>
    </div>

    {/** Demo Stats */}
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
      <StatCard
        title="Products"
        value="-"
        subtitle="Coming Soon"
        icon={<PackageIcon />}
      />
      <StatCard
        title="Sales Today"
        value="N0"
        subtitle="No sales yet"
        icon={<SalesIcon />}
      />
      <StatCard
        title="Stock Items"
        value="-"
        subtitle="Coming Soon"
        icon={<InventoryIcon />}
      />
    </div>

    {/** What's Next */}
    <div className="card p-6">
      <h2 className="text-lg font-semibold mb-4">What's Next?</h2>
      <div className="space-y-3">
        <OnboardingStep
          icon={<PackageIcon />}
          title="Add Products"
          description="Start by adding your first products to the catalog"
          action="Add Products"
          onClick={() => {/** Navigate to products */}}
        />
        <OnboardingStep
          icon={<UsersIcon />}
          title="Invite Staff"
          description="Add team members who can help you manage this branch"
          action="Invite Staff"

```

```

        onClick={() => {/* Navigate to members */}}
      />
      <OnboardingStep
        icon={<SalesIcon />}
        title="Record Your First Sale"
        description="Complete a sale to see how the system works"
        action="Record Sale"
        onClick={() => {/* Navigate to POS */}}
      />
    </div>
  </div>

  {/* Coming Soon Sections */}
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4 mt-6">
    <ComingSoonCard
      title="Recent Sales"
      description="Track your daily sales transactions"
    />
    <ComingSoonCard
      title="Low Stock Alerts"
      description="Get notified when products are running low"
    />
    <ComingSoonCard
      title="Sales Reports"
      description="View detailed sales analytics"
    />
    <ComingSoonCard
      title="Stock Management"
      description="Manage inventory across branches"
    />
  </div>
</div>
);
}

```

User Story 2.2: Branch Switching

As a user with multiple branches, I want to switch between branches so I can manage operations at different locations.

Acceptance Criteria

- [] Branch selector always visible in app header
- [] Dropdown shows all branches in organization
- [] Current branch highlighted
- [] Clicking branch switches context
- [] Page reloads with new branch data
- [] Loading state during switch
- [] Error handling if branch access lost

Technical Tasks

- ☐ Frontend: Branch selector component (already in Story 1.3)
- ☐ State management: update activeBranch in global store
- ☐ Refetch all branch-scoped queries on change
- ☐ Loading spinner during branch switch
- ☐ Error boundary for branch access errors

User Story 2.3: No Branch State (Empty State)

As a user who somehow has no branches, I want to see a helpful error message so I know what to do next.

Acceptance Criteria

- ☐ If user has organization but no branches, show empty state
- ☐ Message: "This organization has no branches yet"
- ☐ Action button: "Create Your First Branch"
- ☐ Clicking opens branch creation modal
- ☐ After creation, redirect to branch dashboard
- ☐ Should not happen in normal flow (branch created during onboarding)

Technical Tasks

- ☐ Frontend: Empty state component
- ☐ Query: check if branches exist
- ☐ Branch creation modal
- ☐ Redirect after creation

Technical Implementation Details

API Endpoints

```
// Branch management (during onboarding)
GET    /v1/workspaces/:workspaceId/branches
POST   /v1/workspaces/:workspaceId/branches
GET    /v1/workspaces/:workspaceId/branches/:branchId
PATCH /v1/workspaces/:workspaceId/branches/:branchId

// Branch dashboard (demo)
GET    /v1/workspaces/:workspaceId/branches/:branchId/dashboard
```

Convex Queries & Mutations

```
// convex/branches.ts

export const getByWorkspace = query({
  args: { workspaceId: v.id("workspaces") },
  handler: async (ctx, args) => {
    return await ctx.db
      .query("branches")
      .withIndex("by_workspace", (q) => q.eq("workspaceId", args.workspaceId))
      .filter((q) => q.eq(q.field("status"), "active"))
      .order("asc")
      .collect();
  }
});

export const getById = query({
  args: { branchId: v.id("branches"), workspaceId: v.id("workspaces") },
  handler: async (ctx, args) => {
    const branch = await ctx.db.get(args.branchId);

    if (!branch || branch.workspaceId !== args.workspaceId) {
      return null;
    }

    return branch;
  }
});

export const create = mutation({
  args: {
    workspaceId: v.id("workspaces"),
    name: v.string(),
    code: v.string(),
    isPrimary: v.boolean(),
  },
  handler: async (ctx, args) => {
    const now = Date.now();

    const branchId = await ctx.db.insert("branches", {
      workspaceId: args.workspaceId,
      name: args.name,
      code: args.code,
      isPrimary: args.isPrimary,
      status: "active",
      createdAt: now,
      updatedAt: now,
    });

    return await ctx.db.get(branchId);
  }
});

export const update = mutation({
  args: {
    branchId: v.id("branches"),
```

```

    name: v.optional(v.string()),
    code: v.optional(v.string()),
  },
  handler: async (ctx, args) => {
    const branch = await ctx.db.get(args.branchId);

    if (!branch) {
      throw new Error("Branch not found");
    }

    await ctx.db.patch(args.branchId, {
      name: args.name,
      code: args.code,
      updatedAt: Date.now(),
    });

    return await ctx.db.get(args.branchId);
  }
});

```

Frontend State Management

```

// stores/branchStore.ts

import { create } from 'zustand';

interface BranchState {
  activeBranch: Branch | null;
  branches: Branch[];
  setActiveBranch: (branch: Branch) => void;
  loadBranches: (workspaceId: string) => Promise<void>;
  createBranch: (data: CreateBranchInput) => Promise<Branch>;
}

export const useBranchStore = create<BranchState>((set, get) => ({
  activeBranch: null,
  branches: [],

  setActiveBranch: (branch) => {
    set({ activeBranch: branch });
  },

  loadBranches: async (workspaceId) => {
    const response = await fetch(`/v1/workspaces/${workspaceId}/branches`);
    const { branches } = await response.json();
    set({ branches });

    // Auto-select primary branch if none selected
    if (!get().activeBranch) {
      const primary = branches.find(b => b.isPrimary);
      if (primary) {
        set({ activeBranch: primary });
      }
    }
  },
}),

```

```
createBranch: async (data) => {
  const response = await fetch(`/v1/workspaces/${data.workspaceId}/branches`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  const { branch } = await response.json();

  // Add to list
  set((state) => ({
    branches: [...state.branches, branch],
    activeBranch: branch, // Auto-select new branch
  }));

  return branch;
},
}));
```

Testing Checklist

Unit Tests

- ☐ Branch CRUD operations work
- ☐ Branch code auto-generation works
- ☐ Primary branch validation works
- ☐ Branch queries return correct data

Integration Tests

- ☐ Branch created during onboarding
- ☐ Branch selector shows all branches
- ☐ Branch switching updates context
- ☐ Dashboard shows correct branch info

E2E Tests

- ☐ Create organization → create branch → see dashboard
- ☐ Create second branch → switch → see new branch dashboard
- ☐ Edit branch name → see updated name in selector

Deployment Plan

Days 1-3: Schema & CRUD

- ☐ Deploy branches table
- ☐ Deploy branch CRUD APIs
- ☐ Update onboarding wizard with branch step
- ☐ Test: create branch during onboarding

Days 4-7: Dashboard & Switching

- ☐ Deploy branch selector component
- ☐ Deploy branch dashboard (demo view)
- ☐ Deploy branch switching logic
- ☐ Test: switch between branches

Success Metrics

- ✓ Users can create branch during onboarding
- ✓ Branch selector shows all branches
- ✓ Branch switching works (< 2 seconds)
- ✓ Dashboard shows correct branch info
- ✓ No errors when accessing branch-scoped routes

Out of Scope (For Now)

- ✗ Stock tracking per branch
- ✗ Sales per branch
- ✗ Product catalog per branch
- ✗ Staff branch assignments
- ✗ Stock transfers between branches
- ✗ Branch-level reports
- ✗ Branch-specific permissions

This implementation plan is ready for development. Focus on branch CRUD, switching, and demo dashboard only. No inventory features yet.